Project: University Management System using OOP

Description:

Create a University Management System that allows administrators to manage students, courses, and faculty members. The system should allow students to enroll in courses, view their course schedules, and check their grades. Faculty members should be able to view their course assignments and student rosters.

Concepts Covered:

1. Class in Python: Define classes for Student, Course, Faculty, and University to represent the different entities in the system.
2. Objects in Python: Create objects for each class to represent individual students, courses, faculty members, and the university.
3. Polymorphism in Python: Implement polymorphic methods for enroll and view_schedule that can be applied to different types of students (e.g., undergraduate, graduate).
4. Encapsulation in Python: Use encapsulation to hide the implementation details of the Student and Faculty classes and provide a public interface for administrators to interact with them.
5. Inheritance in Python: Use inheritance to create a hierarchy of student types (e.g., UndergraduateStudent inherits from Student) and to create a Department class that inherits from University.
6. Data Abstraction in Python: Use data abstraction to represent complex data structures (e.g., course schedules, student rosters) in a simplified way, making it easier to work with the data.

```
# Define classes

class University:

    num_universities = 0  # Class variable

    def __init__(self, name):

        self.name = name

        self.departments = []

        University.num_universities += 1  # Increment class variable

    def add_department(self, department):

        self.departments.append(department)

    @classmethod

    def get_num_universities(cls):

        return cls.num_universities

class Department:

    def __init__(self, name, university):

        self.name = name

        self.university = university

        self.courses = []
```

```
    def add_course(self, course):
        self.courses.append(course)

    @staticmethod
    def get_department_type():
        return "Academic Department"

class Course:
    def __init__(self, course_id, name, credits):
        self.course_id = course_id
        self.name = name
        self.credits = credits
        self.students = []
        self.schedule = {}

    def add_student(self, student):
        self.students.append(student)

    def set_schedule(self, schedule):
        self.schedule = schedule
```

```
class Student:

    def __init__(self, student_id, name, major):

        self.student_id = student_id

        self.name = name

        self.major = major

        self.courses = []



    def enroll(self, course):

        self.courses.append(course)



    def view_schedule(self):

        print("Course Schedule:")

        for course in self.courses:

            print(f"{course.name} ({course.course_id}): {course.credits}
credits")

            for day, time in course.schedule.items():

                print(f"  {day}: {time}")

            print()
```

```
class UndergraduateStudent(Student):

    def __init__(self, student_id, name, major, gpa):

        super().__init__(student_id, name, major)

        self.gpa = gpa



    def view_schedule(self):

        super().view_schedule()



class Faculty:

    def __init__(self, faculty_id, name, department):

        self.faculty_id = faculty_id

        self.name = name

        self.department = department

        self.courses = []



    def assign_course(self, course):

        self.courses.append(course)



    def view_roster(self):

        print("Student Roster:")
```

```
        for course in self.courses:
            print(f"{course.name} ({course.course_id}):")
            for student in course.students:
                print(f"  {student.name} ({student.student_id})")
            print()

    @classmethod
    def get_faculty_type(cls):
        return "Academic Faculty"

# Create objects
university = University("Example University")
department = Department("Computer Science", university)
course1 = Course("CS101", "Introduction to Computer Science", 3)
course1.set_schedule({"Monday": "9:00 AM - 10:30 AM", "Wednesday": "2:00 PM - 3:30 PM"})
course2 = Course("CS202", "Data Structures", 4)
course2.set_schedule({"Tuesday": "10:00 AM - 11:30 AM", "Thursday": "1:00 PM - 2:30 PM"})

student1 = UndergraduateStudent("123456", "John Doe", "Computer Science", 3.5)
```

```
student2 = Student("789012", "Jane Smith", "Mathematics")


faculty1 = Faculty("F123", "Dr. John Smith", department)


# Perform operations
student1.enroll(course1)
student2.enroll(course2)


faculty1.assign_course(course1)
faculty1.view_roster()


university.add_department(department)
department.add_course(course1)
department.add_course(course2)


print("Number of universities:", University.get_num_universities())
print("Department type:", Department.get_department_type())
print("Faculty type:", Faculty.get_faculty_type())
```

This project covers all the concepts and provides a comprehensive example of how to implement them in a real-world scenario.

2.Employee Management System

This project demonstrates the concepts of Inheritance, Polymorphism, and Encapsulation in Python OOPs.

Code:

```
# Employee class (Parent class)
class Employee:
    employee_count = 0  # Class variable

    def __init__(self, name, idnumber, salary):
        self.__name = name  # Private attribute
        self.idnumber = idnumber
        self.salary = salary
        Employee.employee_count += 1  # Increment class variable

    def display(self):
        print("Name:", self.__name)
        print("ID Number:", self.idnumber)
        print("Salary:", self.salary)

    def details(self):
        print("My name is", self.__name)
        print("IdNumber:", self.idnumber)

    @staticmethod
    def get_employee_count():
        return Employee.employee_count

    @classmethod
    def get_employee_count_classmethod(cls):
        return cls.employee_count

    def __str__(self):
        return f"Name: {self.__name}, ID Number: {self.idnumber}, Salary: {self.salary}"

    def __eq__(self, other):
        return self.idnumber == other.idnumber

    def __lt__(self, other):
```

```
        return self.salary < other.salary

    def __gt__(self, other):
        return self.salary > other.salary

# Manager class (Child class of Employee)
class Manager(Employee):
    def __init__(self, name, idnumber, salary, department):
        super().__init__(name, idnumber, salary)
        self.department = department

    def details(self):
        super().details()
        print("Department:", self.department)

    def __str__(self):
        return f"Name: {self._Employee__name}, ID Number: {self.idnumber}, Salary:
{self.salary}, Department: {self.department}"

# Developer class (Child class of Employee)
class Developer(Employee):
    def __init__(self, name, idnumber, salary, programming_languages):
        super().__init__(name, idnumber, salary)
        self.programming_languages = programming_languages

    def details(self):
        super().details()
        print("Programming Languages:", self.programming_languages)

    def __str__(self):
        return f"Name: {self._Employee__name}, ID Number: {self.idnumber}, Salary:
{self.salary}, Programming Languages: {self.programming_languages}"

# Create objects
emp1 = Employee("John Doe", 12345, 50000)
mgr1 = Manager("Jane Smith", 67890, 70000, "Marketing")
dev1 = Developer("Bob Johnson", 34567, 60000, ["Python", "Java", "C++"])

# Call methods
print(emp1)
print(mgr1)
print(dev1)

print()

# Operator overwriting
print("emp1 == mgr1:", emp1 == mgr1)
print("emp1 < mgr1:", emp1 < mgr1)
```

```
print("emp1 > mgr1:", emp1 > mgr1)
```

This project demonstrates:

1. **Inheritance: The Manager and Developer classes inherit from the Employee class.**
2. **Polymorphism: The details() method is overridden in the Manager and Developer classes to provide specific behavior.**
3. **Encapsulation: The __name attribute is private in the Employee class, and can only be accessed through the display() and details() methods.**
4. **Static methods: The get_employee_count() method is a static method that returns the total number of employees. It can be called without creating an instance of the class.**
5. **Class methods: The get_employee_count_classmethod() method is a class method that returns the total number of employees. It can be called without creating an instance of the class, and it has access to the class variables.**
6. **Operator overwriting: The __str__, __eq__, __lt__, and __gt__ methods are overwritten to provide custom behavior for the print(), ==, <, and > operators, respectively.**